

Scheduling Cheat Sheet: Standard Spring Scheduling & Quartz

Standard scheduling (@Scheduled)

@EnableScheduling on a @Configuration . A method has no arguments; it may return a reactive type or be a Kotlin suspend function.

Attribute	Meaning
fixedRate	start-to-start period
fixedDelay	end-to-start gap
initialDelay	wait before the first run
cron + zone	6-field Spring cron, time zone (default: server)
timeUnit	unit of the numeric attributes (ms default)

Use exactly one of fixedRate , fixedDelay , cron . *String variants take \${prop} / ISO-8601 (PT30S).

- State is in memory; lost on restart; **every instance runs every tick**; no misfire handling; no runtime add/remove besides TaskScheduler .
- Boot default:** ThreadPoolTaskScheduler with 1 thread (spring.task.scheduling.pool.size , thread-name-prefix , shutdown.await-termination), so one slow job delays the rest.
- spring.threads.virtual.enabled=true gives a SimpleAsyncTaskScheduler on virtual threads (spring.task.scheduling.simple.concurrency-limit).
- Reactive / Kotlin: the returned Mono / Flux or suspend body is subscribed per run.

ShedLock 7.x (lock and skip)

Deps: net.javacrumbs.shedlock:shedlock-spring + a provider: shedlock-provider-jdbc-template , -mongo , -redis-spring (all 7.10.1).

```
@EnableScheduling
@EnableSchedulerLock(defaultLockAtMostFor="PT30M")
@Scheduled(cron="0 0 2 * * *")
@SchedulerLock(name="nightly",
    lockAtMostFor="PT15M",
    lockAtLeastFor="PT1M")
```

- name : unique per job. lockAtMostFor : crash safety net, longer than the worst run. lockAtLeastFor : minimum hold (clock skew, fast jobs).
- Every node fires; the one without the lock **skips** the tick. No queue, no retry, no misfire handling, schedule still in code.
- One LockProvider bean: JdbcTemplateLockProvider , MongoLockProvider , RedisLockProvider .

Standard (+ ShedLock) vs Quartz

Need	Pick
Fixed rate/delay or a cron, one node, loss on restart is fine	@Scheduled
Several replicas, same code schedule, skip duplicates	@Scheduled + ShedLock
Schedules stored in a DB, survive restarts	Quartz JDBC
Add / pause / reschedule at runtime, per-user jobs	Quartz
Misfire policies, calendars, job data, listeners	Quartz
Fire on one node with fail-over of running jobs	Quartz cluster (JDBC)
Out-of-process, one container per run	Kubernetes CronJob

Quartz costs a schema, a thread pool, DB load and more concepts. Start with @Scheduled .

Runtime API (Scheduler)

```
scheduler.scheduleJob(jd, trigger);
scheduler.addJob(jd, true, true);
// durable
scheduler.triggerJob(jobKey);
// run now
scheduler.pauseJob(k); resumeJob(k);
scheduler.pauseTrigger(tk);
resumeTrigger(tk);
scheduler.rescheduleJob(tk, newTrigger);
scheduler.unscheduleJob(tk);
scheduler.deleteJob(jobKey);
scheduler.checkExists(jobKey);
scheduler.getTriggerState(tk);
// NORMAL
scheduler.standby(); start();
shutdown(true);
```

States: NORMAL PAUSED BLOCKED ERROR COMPLETE NONE . Make dynamic schedules idempotent: replace=true or checkExists . Expose management endpoints only behind authentication.

Quartz model

Scheduler holds JobDetail (what, a Job class + JobDataMap) and Trigger (when). A job can have many triggers; keys are name+group (JobKey , TriggerKey).

```
JobDetail jd =
    JobBuilder.newJob(MyJob.class)

.withIdentity("rem","bks").usingJobData("id",7)

.storeDurably().requestRecovery().build();
Trigger t =
    TriggerBuilder.newTrigger().forJob(jd)

.startNow().withSchedule(CronScheduleBuilder
    .cronSchedule("0 0 2 * * ?"))
    .build();
```

Trigger	Builder
Simple	SimpleScheduleBuilder interval + repeat count
Cron	CronScheduleBuilder
Calendar interval	CalendarIntervalScheduleBuilder (months, DST-safe days)
Daily time interval	DailyTimeIntervalScheduleBuilder (window, days)

- @DisallowConcurrentExecution : one run per JobDetail at a time.
- @PersistJobDataAfterExecution : write the map back (use with the first).
- JobExecutionException (refireImmediately , unscheduleFiringTrigger , unscheduleAllTriggers) ; InterruptableJob ; Calendar excludes times.
- JDBC JobDataMap : store ids and primitives, not entities.

Cron dialects

	Spring	Quartz	Unix / K8s
Fields	6 (sec first)	6 + optional year	5 (no sec)
Day-of-week	0-7, 0 and 7 = Sun, SUN - SAT	1-7, 1 = Sun	0-7, 0 and 7 = Sun
?	allowed	required in one of DoM / DoW	no
L W #	yes	yes (L-3, 15W, 6#3)	no
Macros	@daily ...	no	some
Zone	zone=	inTimeZone	CronJob timeZone

Specials: * any, , list, - range, / step (no wrap), ? no value, L last, W nearest weekday (never crosses a month), # nth weekday.
Spring: 0 0 2 * * MON-FRI ; Quartz: 0 0 2 ? * MON-FRI . DST: a time that does not exist is skipped or shifted, test it.

Observability

- management.endpoints.web.exposure.include=quartz,scheduledtasks ; Quartz is not exposed by default; management.endpoint.quartz.access : values sanitized (show-values).
- GET /actuator/quartz , /jobs , /triggers , /jobs/{group}/{name} . POST /actuator/quartz/jobs/{group}/{name} with {"state":"running"} triggers now.
- /actuator/scheduledtasks lists @Scheduled tasks; observation tasks.scheduled.execution (Micrometer metrics and traces).
- No built-in Quartz meters or health: add a JobListener / TriggerListener with a Timer and a Counter (tag job, outcome) and a HealthIndicator .
- MDC per run for logs; alert on missed fires (nextFireTime in the past, state ERROR).

Misfires

Late by more than misfireThreshold : **RAM 5000 ms** (Boot default), **JDBC 60000 ms**; maxMisfiresToHandleAtATime 20 (JDBC). Constants are Trigger.MISFIRE_INSTRUCTION_* (prefix omitted). Smart policy = 0, default of every builder.

- Simple, once:** smart = FIRE_NOW ; also IGNORE_MISFIRE_POLICY .
- Simple, forever:** smart = RESCHEDULE_NEXT_WITH_REMAINING_COUNT ; also RESCHEDULE_NEXT_WITH_EXISTING_COUNT , RESCHEDULE_NOW_WITH_REMAINING_REPEAT_COUNT , RESCHEDULE_NOW_WITH_EXISTING_REPEAT_COUNT , FIRE_NOW , IGNORE_MISFIRE_POLICY .
- Simple, finite:** smart = RESCHEDULE_NOW_WITH_EXISTING_REPEAT_COUNT ; same others.
- Cron:** smart = FIRE_ONCE_NOW ; also DO_NOTHING , IGNORE_MISFIRE_POLICY .
- Calendar / daily interval:** smart = FIRE_ONCE_NOW ; also DO_NOTHING , IGNORE_MISFIRE_POLICY .

Set with withMisfireHandlingInstruction...(). Causes: app down, standby, busy threads, blocked job, paused trigger. @Scheduled has no misfires: missed ticks are lost.

Spring Boot integration

spring-boot-starter-quartz (test: spring-boot-starter-quartz-test , Boot 4 only). Auto-registers JobDetail , Trigger , Calendar beans and one Scheduler ; jobs extend QuartzJobBean (executeInternal) and get setter injection from the JobDataMap and the context.

Property	Default
spring.quartz.job-store-type	memory
.scheduler-name	bean name quartzScheduler
.auto-startup	true
.startup-delay	0s (whole seconds)
.wait-for-jobs-to-complete-on-shutdown	false
.overwrite-existing-jobs	false
.properties.*	empty, passed as org.quartz.*
.jdbc.initialize-schema	embedded (always drops tables!)
.jdbc.schema / .platform	tables_@platform @.sql / detected
.jdbc.comment-prefix	#, --

- Thread pool: SimpleThreadPool , threadCount 10. Instance id NON_CLUSTERED .
- A quartz.properties on the classpath is ignored: use spring.quartz.properties.* or a customizer.
- @QuartzDataSource / @QuartzTransactionManager pick a dedicated pool; SchedulerFactoryBeanCustomizer runs last.

Testing and anti-patterns

- Unit-test job logic without a scheduler; inject a Clock . Test trigger definitions (next fire times, misfire instruction).
- @SpringBootTest with spring.quartz.auto-startup=false , or a unique scheduler-name ; Awaitility, not sleep .
- One Testcontainers test per database; a two-scheduler cluster test.

Anti-patterns: initialize-schema=always in production; entities in the JobDataMap ; clustered and non-clustered nodes on one schema; relying on a classpath quartz.properties ; heavy work on the 1-thread @Scheduled pool; unsynchronized clocks; non-idempotent recoverable jobs; Terracotta or skipUpdateCheck from old docs.

Databases and stores

Database	Delegate, DDL script, clustering
PostgreSQL	PostgreSQLDelegate (mandatory); tables_postgres ; cluster measured
MySQL, MariaDB	StdJDBCDelegate ; tables_mysql_innoDB (never tables_mysql) ; cluster measured
Oracle	StdJDBCDelegate or oracle.OracleDelegate ; tables_oracle23 with the 23.x driver (set jdbc.schema) ; cluster measured
SQL Server	MSSQLDelegate (mandatory); tables_sqlServer , edit it: drop USE , GO becomes ; ; cluster measured
H2, HSQLDB, Derby	StdJDBCDelegate , HSQLDBDelegate ; tables_h2 , _hsqldb , _derby ; dev and tests, not clustered
DB2, Sybase, CUBRID, GaussDB	DB2v8Delegate (v6, v7), SybaseDelegate , CUBRIDDelegate , GaussDBDelegate ; official scripts; not run
MongoDB	community quartz-mongodb 2.2.0-rc2, no DDL; nothing fires on 2.5.x
Redis	community quartz-redis-jobstore 1.2.0, no DDL; broken on 2.5.x

- JDBC store: LocalDataSourceJobStore (Boot, job-store-type=jdbc), tables QRTZ_* , row locks in QRTZ_LOCKS . Terracotta is gone since 2.4.0.
- Schema: embedded (in-memory DBs only), never (production, Flyway/Liquibase), always (only throwaway DBs).
- tablePrefix renames the tables; org.quartz.jobStore.useProperties=true keeps map values as strings.

Clustering (JDBC only)

Property	Default
jobStore.isClustered	false
jobStore.clusterCheckinInterval	7500 ms
scheduler.instanceId	NON_CLUSTERED , use AUTO
scheduler.dbFailureRetryInterval	15000 ms
jobStore.acquireTriggersWithinLock	false
scheduler.batchTriggerAcquisitionMaxCount	1

- Same DB, same scheduler-name, unique instance id, **clocks synchronized** (NTP). Never mix clustered and non-clustered nodes on one schema.
- Any node fires a trigger once; **no load balancing**. A dead node is detected after about a check-in interval plus margin; its jobs with requestRecovery run again elsewhere, so make them idempotent.